

ESP32 PATIENT HEALTH MONITOR - ARDUINO C++ FIRMWARE

Complete Source Code: AD8232 ECG + MAX30102 SpO2/HR + DS18B20 Temp + REST Telemetry

FORMAT: SIMPLE MONOCHROME PRINTABLE SPECIFICATION | TARGET: ESP32 DEVKIT V1

1. HARDWARE PIN ASSIGNMENT & BUS SPECIFICATION

Sensor / Device	Sensor Pin	ESP32 GPIO Pin	Bus / Protocol	Electrical Notes
AD8232 ECG Module	OUTPUT	GPIO 36 (VP)	Analog ADC1_CH0	12-bit ADC (0-4095), centered at 1.65V
AD8232 ECG Module	L0+ (Leads Off +)	GPIO 2	Digital Input	Lead detachment detection (Active HIGH)
AD8232 ECG Module	L0- (Leads Off -)	GPIO 4	Digital Input	Lead detachment detection (Active HIGH)
MAX30102 Biosensor	SDA	GPIO 21	I2C Data Bus	Internal 3.3V logic with 400kHz fast mode
MAX30102 Biosensor	SCL	GPIO 22	I2C Clock Bus	Connected with internal pull-ups
DS18B20 Temp Probe	DQ (Data)	GPIO 15	Dallas 1-Wire	Requires 4.7 kOhm pull-up resistor to 3.3V
Emergency Buzzer / LED	Positive Anode	GPIO 13	Digital Output	Active HIGH when CRITICAL triage alert fires

2. REQUIRED ARDUINO LIBRARIES

Library Name	Author / Source	Version	Usage in Firmware
WiFi.h & HTTPClient.h	Espressif Systems	Built-in	Wi-Fi connection & REST POST telemetry transmission
Wire.h	Arduino Core	Built-in	I2C communication with MAX30102 pulse oximeter
SparkFun MAX3010x	SparkFun Electronics	v1.1.2+	MAX30102 optical sensor setup & FIFO reading
OneWire	Paul Stoffregen	v2.3.7+	1-Wire bitbang communication protocol
DallasTemperature	Miles Burton	v3.9.0+	DS18B20 digital Celsius conversion

3. COMPLETE FIRMWARE SOURCE CODE (ESP32_HEALTH_MONITOR.INO)

```
001  /*
002   * ESP32 Patient Health Monitor: ECG (AD8232), SpO2 & Heart Rate (MAX30102), Temp ...
003   * Integrates with Secure Health Monitoring Machine Learning Pipeline.
004   *
005   * Hardware:
006   *   - ESP32 NodeMCU / Dev Module
007   *   - AD8232 ECG Sensor (OUT -> GPIO 36, L0+ -> GPIO 2, L0- -> GPIO 4)
008   *   - MAX30102 Pulse Oximeter (SDA -> GPIO 21, SCL -> GPIO 22)
009   *   - DS18B20 Temperature Sensor (DATA -> GPIO 15, with 4.7k pullup)
010   *   - Status LED / Buzzer -> GPIO 13 (Triage Alarm)
011   *
012   * Target API: POST http://<SERVER_IP>:8000/api/esp32/telemetry
013   */
014
015  #include <WiFi.h>
016  #include <HTTPClient.h>
017  #include <Wire.h>
018  #include "MAX30105.h"           // SparkFun MAX3010x library
019  #include "heartRate.h"
020  #include <OneWire.h>
021  #include <DallasTemperature.h>
022
023  // =====
024  // CONFIGURATION & NETWORK CREDENTIALS
025  // =====
026  const char* WIFI_SSID      = "YOUR_WIFI_SSID";
027  const char* WIFI_PASSWORD = "YOUR_WIFI_PASSWORD";
028
029  // IP Address of the machine running the Python Health Pipeline server
030  const char* SERVER_URL     = "http://192.168.1.100:8000/api/esp32/telemetry";
031  const char* AUTH_TOKEN     = "Bearer health_sec_token_9f83a2c0918bd47e";
032  const char* DEVICE_ID      = "ESP32-CLINICAL-NODE-01";
```

```
033
034 // Pin Definitions
035 #define PIN_ECG_IN      36    // VP / ADC1_CH0 (AD8232 OUT)
036 #define PIN_LO_PLUS     2     // AD8232 Leads-Off +
037 #define PIN_LO_MINUS    4     // AD8232 Leads-Off -
038 #define PIN_TEMP_BUS    15    // DS18B20 One-Wire bus
039 #define PIN_ALARM_LED   13    // Clinical critical triage LED / Buzzer
040
041 // Sampling Configuration
042 #define ECG_FS           125   // Sampling frequency (Hz)
043 #define WINDOW_SAMPLES   250  // 2-second telemetry window (250 samples at 125 Hz)
044
045 // =====
046 // GLOBAL INSTANCES & BUFFERS
047 // =====
048 MAX30105 particleSensor;
049 OneWire oneWire(PIN_TEMP_BUS);
050 DallasTemperature tempSensors(&oneWire);
051
052 float ecgBuffer[WINDOW_SAMPLES];
053 int sampleIndex = 0;
054 unsigned long lastSampleMicros = 0;
055 const unsigned long sampleIntervalMicros = 1000000 / ECG_FS; // 8000 us (125 Hz)
056
057 float lastBPM = 72.0;
058 float lastSpO2 = 98.0;
059 float lastTemperatureC = 36.6;
060 bool leadsOff = false;
061
062 // =====
063 // SETUP
064 // =====
065 void setup() {
066     Serial.begin(115200);
067     pinMode(PIN_LO_PLUS, INPUT);
068     pinMode(PIN_LO_MINUS, INPUT);
069     pinMode(PIN_ALARM_LED, OUTPUT);
070     digitalWrite(PIN_ALARM_LED, LOW);
071
072     // ADC resolution
073     analogReadResolution(12); // 0 - 4095
074
075     Serial.println("\n[ESP32 Health Monitor] Initializing...");
076
077     // 1. Connect to Wi-Fi
078     WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
079     Serial.print("Connecting to Wi-Fi: ");
080     Serial.println(WIFI_SSID);
081     while (WiFi.status() != WL_CONNECTED) {
082         delay(500);
083         Serial.print(".");
084     }
085     Serial.println("\nWiFi connected! IP address: ");
086     Serial.println(WiFi.localIP());
087
088     // 2. Initialize MAX30102 Sensor
089     if (particleSensor.begin(Wire, I2C_SPEED_FAST)) {
090         Serial.println("MAX30102 pulse oximeter detected.");
091         particleSensor.setup();
092         particleSensor.setPulseAmplitudeRed(0x0A);
093         particleSensor.setPulseAmplitudeGreen(0);
094     } else {
095         Serial.println("WARNING: MAX30102 not found. Check I2C wiring (SDA: 21, SCL: 2...");
096     }
097
098     // 3. Initialize DS18B20 Temperature Sensor
099     tempSensors.begin();
100     Serial.println("DS18B20 temperature sensor initialized.");
101
102     lastSampleMicros = micros();
103     Serial.println("System online. Streaming physiological signals...");
```

```

104 }
105
106 // =====
107 // MAIN LOOP
108 // =====
109 void loop() {
110     // 1. Check Leads-Off condition on AD8232
111     if ((digitalRead(PIN_LO_PLUS) == 1) || (digitalRead(PIN_LO_MINUS) == 1)) {
112         leadsOff = true;
113     } else {
114         leadsOff = false;
115     }
116
117     // 2. Sample AD8232 ECG at fixed sample rate (125 Hz)
118     unsigned long nowMicros = micros();
119     if (nowMicros - lastSampleMicros >= sampleIntervalMicros) {
120         lastSampleMicros += sampleIntervalMicros;
121
122         if (leadsOff) {
123             ecgBuffer[sampleIndex] = 0.0; // Flatline if electrodes detached
124         } else {
125             int rawADC = analogRead(PIN_ECG_IN);
126             // Convert 12-bit ADC (0-4095 at 3.3V) to approximate physical millivolts
127             // AD8232 output centered at 1.65V with gain ~ 1100
128             float voltageV = (rawADC / 4095.0) * 3.3;
129             float ecg_mV = (voltageV - 1.65) * 1000.0 / 1100.0;
130             ecgBuffer[sampleIndex] = ecg_mV;
131         }
132
133         sampleIndex++;
134
135         // When full buffer window is gathered, transmit to ML Pipeline
136         if (sampleIndex >= WINDOW_SAMPLES) {
137             // Sample vitals
138             readVitals();
139             // Send telemetry packet over Wi-Fi
140             transmitTelemetryPacket();
141             sampleIndex = 0;
142         }
143     }
144
145     // Read MAX30102 infrared & red LEDs for heartbeats
146     long irValue = particleSensor.getIR();
147     if (checkForBeat(irValue)) {
148         // Heartbeat detected
149     }
150 }
151
152 // =====
153 // READ AUXILIARY VITALS
154 // =====
155 void readVitals() {
156     // Read DS18B20
157     tempSensors.requestTemperatures();
158     float temp = tempSensors.getTempCByIndex(0);
159     if (temp > 20.0 && temp < 45.0) {
160         lastTemperatureC = temp;
161     }
162
163     // Read MAX30102 SpO2 & Heart Rate
164     long ir = particleSensor.getIR();
165     long red = particleSensor.getRed();
166     if (ir > 50000) { // Finger detected on sensor
167         float ratio = (float)red / (float)ir;
168         float spo2Est = 110.0 - 25.0 * ratio; // Standard linear calibration approxima...
169         lastSpO2 = constrain(spo2Est, 85.0, 100.0);
170     }
171 }
172
173 // =====
174 // TRANSMIT TELEMETRY TO ML PIPELINE

```

```
175 // =====
176 void transmitTelemetryPacket() {
177     if (WiFi.status() != WL_CONNECTED) {
178         Serial.println("Wi-Fi disconnected. Reconnecting...");
179         return;
180     }
181
182     HTTPClient http;
183     http.begin(SERVER_URL);
184     http.addHeader("Content-Type", "application/json");
185     http.addHeader("Authorization", AUTH_TOKEN);
186
187     // Build JSON payload
188     String json = "{";
189     json += "\"device_id\": \"" + String(DEVICE_ID) + "\", ";
190     json += "\"fs\": " + String(ECG_FS) + ", ";
191     json += "\"leads_off\": " + String(leadsOff ? "true" : "false") + ", ";
192     json += "\"bpm\": " + String(lastBPM, 1) + ", ";
193     json += "\"spo2\": " + String(lastSpO2, 1) + ", ";
194     json += "\"temperature_c\": " + String(lastTemperatureC, 2) + ", ";
195     json += "\"signal\":";
196
197     for (int i = 0; i < WINDOW_SAMPLES; i++) {
198         json += String(ecgBuffer[i], 3);
199         if (i < WINDOW_SAMPLES - 1) json += ", ";
200     }
201     json += "]}";
202
203     // POST Request
204     int httpResponseCode = http.POST(json);
205
206     if (httpResponseCode == 200) {
207         String response = http.getString();
208         Serial.print("Pipeline Response: ");
209         Serial.println(response);
210
211         // Check for CRITICAL triage alarm from ML model
212         if (response.indexOf("\"triage_risk_tier\": \"CRITICAL\"") > 0) {
213             digitalWrite(PIN_ALARM_LED, HIGH); // Alarm LED ON
214             Serial.println(">>> CLINICAL ALERT: CRITICAL CARDIAC EVENT DETECTED! <<<");
215         } else {
216             digitalWrite(PIN_ALARM_LED, LOW); // Normal
217         }
218     } else {
219         Serial.print("HTTP POST failed, code: ");
220         Serial.println(httpResponseCode);
221     }
222
223     http.end();
224 }
```